

JobAgent

Project Structure & Codebase Analysis

A personal job-search and job-application assistant

Owner: Meet Palan

Prepared: 2026-09-13

1. Overview

JobAgent is a locally-run, personal job-search assistant organized around a linear pipeline: discover jobs, score/shortlist them, apply, then track outcomes. It has no server backend and no database -- all state lives in plain JSON files and Markdown notes inside this repository, read and written by small, single-purpose Python scripts.

The project is intentionally minimal by design (see CLAUDE.md). There is currently no web scraping of login-gated sites, no browser automation, and no automatic submission of applications. Only public, unauthenticated ATS/career-site JSON APIs are queried (Greenhouse, Lever, Ashby, SmartRecruiters, and Workday's "cxs" endpoint).

Pipeline at a glance

```
discover_jobs.py --> score_job.py --> data/jobs.json --> jobs/shortlisted/*.md
(fetch postings)   (match & rank)   (source of truth)   (human-readable notes)

                        |
                        v
                    dashboard.py (local web UI, read-only)
                        |
                        v
human reviews --> applications/* --> data/applications.json
```

Design principles observed in the code

- High recall, then filter by score: discovery never drops a job for a missing field; scripts/score_job.py alone decides fit.
- Score and "should I apply" are computed independently -- a high numeric score can still be SKIP (e.g. a senior-level title).
- Evidence is never inflated: "professional" (paid work) is always distinguished from "practical" (projects/courses/tools) in the candidate profile and in scoring.
- Missing data is treated as "unknown", never as a negative signal (location, salary, industry, work mode).
- Every write path is idempotent/deduplicating: rediscovering the same posting updates last_seen_at rather than duplicating it.
- The dashboard is strictly read-only: it never calls an LLM, never re-scores, never writes to data/jobs.json.

2. Folder Structure

```

JobAgent/
|-- CLAUDE.md           Guidance for Claude Code working in this repo
|-- README.md           Project overview & structure (developer-facing)
|-- .gitignore           Excludes personal docs, caches, env files
|-- dashboard.py         Local read-only HTTP server + JSON API for jobs.json
|
|-- profile/            The user's own information, maintained by hand
|   |-- profile.md       Contact info, target roles, experience, education
|   |-- skills.md        Skills inventory & certifications
|   |-- preferences.md   Search preferences (location, salary, industries)
|   |-- resume/          Source resume PDF (gitignored -- contains PII)
|
|-- jobs/               Postings move between these 4 states as folders
|   |-- discovered/      Newly found, not yet reviewed (.gitkeep only)
|   |-- shortlisted/     14 .md notes -- worth applying to (AUTO_APPLY/REVIEW)
|   |-- rejected/        Passed on / employer-rejected (.gitkeep only)
|   |-- applied/         Already applied to (.gitkeep only)
|
|-- applications/        Generated application material, per-application
|   |-- resumes/         Tailored resume versions (gitignored)
|   |-- cover_letters/   Tailored cover letters (gitignored)
|   |-- answers/         Saved answers to recurring application questions
|   |-- pending/         Drafted but not yet submitted
|
|-- data/               Structured, machine-readable source of truth
|   |-- jobs.json        139 tracked postings (see section 5)
|   |-- applications.json Tracked applications & status (currently empty: [])
|   |-- candidate_profile.json Structured mirror of profile/*.md, used by scorer
|   |-- discovery_sources.json Configured companies per ATS connector
|
|-- scripts/            Small, single-purpose automation scripts
|   |-- discover_jobs.py  Fetch + normalize + dedupe + score + save postings
|   |-- score_job.py      Standalone matching/scoring engine
|
|-- dashboard/          Static HTML/JS front-end for dashboard.py (330 lines)
|
|-- tests/              One test file per script (1,320 lines total)
|   |-- discover_jobs.py  693 lines
|   |-- score_job.py      447 lines
|   |-- dashboard.py      180 lines
|
|-- docs/               Empty (.gitkeep only) -- reserved for future docs

```

3. Data Model

data/jobs.json -- tracked postings (source of truth)

Every fetched/scored posting is stored as one JSON object. Key fields:

```
id, company, title, location, work_mode, industry, company_size,
salary_min/max/currency, min/max_experience_years,
required_skills[], preferred_skills[], description,
application_url, source_url, source_job_id, source,
posted_date, date_discovered, first_seen_at, last_seen_at,
score, role_family, career_level,
application_decision (AUTO_APPLY | REVIEW | SKIP), decision_reason,
score_breakdown{...}, experience_gap_years,
relevant_evidence[], transferable_evidence[], missing_requirements[],
experience_gaps[], risks[], status (discovered | shortlisted)
```

data/applications.json -- tracked applications

Currently an empty array ([]) -- no applications have been submitted yet. Once populated, each entry is expected to record the application's job_id / application_url plus status, so scripts/score_job.py's verify_auto_apply_preconditions() can detect already-applied jobs.

data/candidate_profile.json -- structured mirror of profile/*.md

Machine-readable input consumed by both scripts. It carries target/secondary titles, skills, location/salary/industry preferences, and a `capabilities` block -- one entry per skill area (business_analysis, product_management, project_management, analytics, etc.), each rated 'professional' (paid work), 'practical' (projects/courses/tools), or 'none', with evidence quotes pulled straight from the resume. This strength rating is what the scorer uses to avoid crediting project work as if it were job experience.

data/discovery_sources.json -- configured ATS connectors

Connector	Companies configured
Greenhouse	12 (groww, postman, highradius, netskope, druva, zetaglobal, gitlab, justworks, figma, discord, robinhood,
Lever	3 (zeta, meesho, cred)
Ashby	8 (ramp, notion, linear, vercel, replit, watershed, mercury, openai)
SmartRecruiters	0 (connector built but no company has its public feed enabled -- see note below)
Workday	1 (Accenture)

A documented "_unavailable_sources" block explains why LinkedIn, Naukri, Indeed, and Wellfound are deliberately excluded -- each either blocks automated/AI access in robots.txt, prohibits scraping in its Terms of Service, or has no public bulk API.

4. Scripts

[scripts/discover_jobs.py](#)

Fetches postings from every configured source, filters by the candidate's expanded target titles and preferred locations, normalizes each into one common schema, deduplicates against both this run and data/jobs.json, scores new unique jobs via score_job.py, then saves everything back to data/jobs.json (writing a Markdown note under jobs/shortlisted/ for anything scoring AUTO_APPLY or REVIEW). Supports --dry-run to preview without writing, and per-source safety caps (max pages/jobs per source, request delay/timeout) so one broken source can't run unbounded.

- Deduplication is confidence-tiered: exact (source, source_job_id) match first, then canonical application URL, then (company, title, location, required_skills).
- Title expansion is curated per role family (product / business_analysis / project) -- never expands into unrelated families like engineering or sales.
- A missing location always passes the location filter; only a location stated AND clearly outside the preferred list is excluded.

[scripts/score_job.py](#)

A personal (not general-purpose) matching engine. It classifies the job's role family and career level from the title first, scores nine weighted components, then independently decides AUTO_APPLY / REVIEW / SKIP using hard gates -- so a high score alone can never auto-qualify a senior-level or wrong-role-family posting.

Score component	Weight	Notes
Role family match	20	Dampened to 20% if job's family isn't a target family
Capability relevance	20	Dampened by role-family mismatch
Relevant evidence	15	Phrase coverage of required/nice-to-have skills
Title alignment	10	Exact / secondary / same-family / partial-word match
Experience fit	10	Gradient by gap size; senior roles scored conservatively
Career level fit	10	Entry/junior favored; mid/senior/lead scored down
Location / work mode	8	Unknown location or mode never penalized
Industry / company size	4	Unknown never penalized; avoided industries zeroed
Salary fit	3	Unknown never penalized

Decision thresholds: AUTO_APPLY needs score ≥ 85 , entry/junior level, ≤ 1 yr experience gap, no critical gaps, and a legitimate-looking listing. REVIEW is score ≥ 75 (or an otherwise-passing job with some soft concern). Anything below, or hitting a hard SKIP gate (senior+ title, role-family mismatch, > 2 yr experience gap, or a critical missing requirement), is SKIP.

[dashboard.py + dashboard/index.html](#)

A zero-dependency local HTTP server (Python standard library only) serving one static page and a /api/jobs endpoint. It reads data/jobs.json fresh on every request, supports filtering (decision, role family, location, source, minimum score, free-text search) and sorting (score, newest, company, title), and is explicitly documented as display-only: no LLM calls, no re-scoring, no writes.

5. Current Data Snapshot

Live counts read from data/jobs.json and data/applications.json at analysis time.

Jobs by application decision (139 total tracked postings)

Decision	Count	Meaning
SKIP	124	Hard-gated out (wrong role family, senior level, large exp. gap, or low score)
REVIEW	14	Worth a human look; written to jobs/shortlisted/*.md
AUTO_APPLY	1	Meets every auto-apply condition; also shortlisted

Jobs by source connector

Source	Count
Workday (Accenture)	92
Greenhouse	42
Lever	4
Ashby	1

Applications

data/applications.json currently contains an empty array ([]) -- no applications have been submitted yet. jobs/applied/ and jobs/rejected/ each hold only a .gitkeep placeholder.

Shortlisted notes on disk

15 jobs are marked status="shortlisted" in jobs.json; 14 corresponding Markdown files exist under jobs/shortlisted/, covering Druva (Product Manager), Meesho (Product Manager, Internal Platform), and 12 Accenture (Workday) postings spanning Business Analyst, Business Advisory Analyst, Product Management Analyst, and Program/Project Management roles.

6. Candidate Profile Summary

Name	Meet Palan
Location	Rajkot, Gujarat, India
Current role	Business Analyst and Project Coordinator, Tibicle LLP (Apr 2025-Present)
Total experience	1+ years
Target titles	Business Analyst, Project Manager, Product Manager, Associate Product Manager
Secondary title	Project Coordinator
Preferred locations	Ahmedabad, GIFT City / Gandhinagar, Pune, Bangalore, Mumbai
Work mode	Open to remote, hybrid, or onsite (max 2h commute)
Salary range	Min INR 3,00,000/yr; target INR 3.5-4 LPA
Earliest start date	2 October 2026
Education	B.Tech Computer Engineering, Marwadi University (CGPA 9.44, 2022-2025)

Evidence-backed capability strengths (from data/candidate_profile.json)

Capability	Strength
Business analysis	Professional
Requirements gathering	Professional
Stakeholder communication	Professional
Documentation (BRD/SOW)	Professional
Project management	Professional
Agile / Scrum	Professional
Client-facing	Professional
Product management	Practical (certification + project work only)
Analytics (SQL/Tableau/Excel)	Practical
Software development	Practical
Design (Figma) / QA testing / founder work	None

7. Conventions & Guardrails (from CLAUDE.md)

- data/jobs.json and data/applications.json are the sources of truth; files under jobs/*/ and applications/*/ are human-readable working artifacts tied to entries in those JSON files.
- Jobs move between discovered -> shortlisted -> rejected -> applied as four states of the same entry, not four independent folders.
- Generated application documents are named per-application (not per-job), since one job may get multiple drafts.
- profile/resume/ and generated resumes/cover letters are gitignored (personal information) and must not be removed from .gitignore without being asked.
- Scripts under scripts/ are kept small and single-purpose, each with a corresponding test file under tests/.
- Before adding scraping, browser automation, or auto-submission features, scope must be confirmed with the user first -- these touch external sites and submit real applications on the user's behalf.

8. Test Coverage

Test file	Lines	Covers
tests/discover_jobs.py	693	HTML stripping, experience/salary parsing, title/location relevance, per-connector normalization
tests/score_job.py	447	Role-family & career-level classification, each scoring component, AUTO_APPLY/REVIEW/STATUS
tests/dashboard.py	180	load_jobs tolerance, filter/sort logic, payload counts

9. Gaps & Suggested Next Steps

- Fill in profile/preferences.md's open item ("anything else the agent should know") if any additional filtering rules exist.
- No applications have been submitted yet -- data/applications.json is empty and jobs/applied/, jobs/rejected/ are unused.
- SmartRecruiters connector is implemented but dormant: no configured company currently exposes a public postings feed.
- docs/ is empty and reserved for future documentation.
- Per CLAUDE.md, any move toward browser automation or auto-submission requires explicit scope confirmation with the user before implementation.